

Assignment 12.4

Name: Navya Revuri

HTNO: 2303A52483

Batch: 44

Task 1: Bubble Sort

Prompt:

Implement Bubble Sort in Python to sort student scores.
Add inline comments for comparisons, swaps, and passes.
Include early termination logic.
Explain best, average, and worst-case time complexity.
Show sample input and output.

```

Assignment 12.1.py > ...
1  #Implement Bubble Sort in Python to sort student scores.
2  #Add inline comments for comparisons, swaps, and passes.
3  #Include early termination logic.
4  #Explain best, average, and worst-case time complexity.
5  #Show sample input and output.
6  # Bubble Sort implementation to sort student scores
7  def bubble_sort(scores):
8      n = len(scores)
9      # Early termination flag
10     swapped = True
11     # Pass counter
12     passes = 0
13     while swapped:
14         swapped = False
15         passes += 1
16         # Inner loop for comparisons and swaps
17         for i in range(n - 1):
18             # Compare adjacent elements
19             if scores[i] > scores[i + 1]:
20                 # Swap elements if they are in wrong order
21                 scores[i], scores[i + 1] = scores[i + 1], scores[i]
22                 swapped = True
23     print(f"Number of passes: {passes}")
24     # Sample input: list of student scores
25     student_scores = [88, 75, 92, 85, 79]
26     print("Original scores:", student_scores)
27     # Sort the scores using bubble sort
28     bubble_sort(student_scores)
29     print("Sorted scores:", student_scores)
30     # Time Complexity Explanation:
31     # Best Case: O(n) - This occurs when the input list is already sorted. The algorithm
32     # Average Case: O(n^2) - This occurs when the input list is in random order. The
33     # Worst Case: O(n^2) - This occurs when the input list is sorted in reverse order
34

```

```

> ▼ TERMINAL
● PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scripts/Activate.ps1"
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scripts/python.exe"
Assit/Assignment 12.1.py"
● Original scores: [88, 75, 92, 85, 79]
  Number of passes: 4
  Sorted scores: [75, 79, 85, 88, 92]
○ (.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>

```

Observation:

Observation

- Bubble Sort compares adjacent elements and swaps them if needed.
- Early termination reduces unnecessary passes when the list is already sorted.

- For small lists (like student scores), it works fine.
- But for large datasets, it becomes slow because time complexity is $O(n^2)$.
- Best case happens when the list is already sorted $\rightarrow$ only 1 pass $\rightarrow O(n)$.

Task 2: Nearly Sorted Data

Prompt:

Write Bubble Sort for nearly sorted roll numbers.

Suggest a better algorithm and implement Insertion Sort.

Explain why Insertion Sort is better for nearly sorted data.

Compare their behaviour with example input/output.

```

Assignment 12.1.py > ...
38 # Bubble Sort implementation for nearly sorted roll numbers
39 def bubble_sort_nearly_sorted(roll_numbers):
40     n = len(roll_numbers)
41     swapped = True
42     passes = 0
43     while swapped:
44         swapped = False
45         passes += 1
46         for i in range(n - 1):
47             if roll_numbers[i] > roll_numbers[i + 1]:
48                 roll_numbers[i], roll_numbers[i + 1] = roll_numbers[i + 1], roll_numbers[i]
49                 swapped = True
50     print(f"Bubble Sort Passes: {passes}")
51 # Insertion Sort implementation for nearly sorted roll numbers
52 def insertion_sort(roll_numbers):
53     n = len(roll_numbers)
54     for i in range(1, n):
55         key = roll_numbers[i]
56         j = i - 1
57         # Move elements of roll_numbers[0..i-1], that are greater than key,
58         # to one position ahead of their current position
59         while j >= 0 and roll_numbers[j] > key:
60             roll_numbers[j + 1] = roll_numbers[j]
61             j -= 1
62         roll_numbers[j + 1] = key
63 # Sample input: nearly sorted roll numbers
64 roll_numbers = [101, 102, 103, 105, 104, 106]
65 print("Original roll numbers:", roll_numbers)
66 # Sort using Bubble Sort
67 bubble_sort_nearly_sorted(roll_numbers.copy())
68 print("Bubble Sorted roll numbers:", roll_numbers)
69 # Sort using Insertion Sort
70 insertion_sort(roll_numbers.copy())
71 print("Insertion Sorted roll numbers:", roll_numbers)
72 # Explanation:
73 # Insertion Sort is better for nearly sorted data because it can quickly place elements in their correct position with fewer comparisons

```

```

SyntaxError: '(' was never closed
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scripts/python.exe" "c:/Users/navya/OneDrive/Desktop/AI Assit/Assignment 12.1.py"
Original roll numbers: [101, 102, 103, 105, 104, 106]
Bubble Sort Passes: 2
Bubble Sorted roll numbers: [101, 102, 103, 105, 104, 106]
Insertion Sorted roll numbers: [101, 102, 103, 105, 104, 106]
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>

```

Observation

- Bubble Sort still performs multiple comparisons even if the list is almost sorted.
- Insertion Sort performs fewer shifts when elements are already near their correct position.
- For nearly sorted roll numbers, Insertion Sort

Task 3: Searching Records

prompt

Implement Linear Search (unsorted data) and Binary Search (sorted data) in Python.

Add docstrings explaining parameters and return values.

Explain when Binary Search is applicable.

Compare time complexity and show sample input/output.

```

74 #Implement Linear Search (unsorted data) and Binary Search (sorted data) in Python.
75 #Add docstrings explaining parameters and return values.
76 #Explain when Binary Search is applicable.
77 #Compare time complexity and show sample input/output.
78 # Linear Search implementation for unsorted data
79 def linear_search(arr, target):
80     """
81     Perform linear search on an unsorted array.
82
83     Parameters:
84     arr (list): The list of elements to search through.
85     target: The element to search for.
86
87     Returns:
88     int: The index of the target element if found, otherwise -1.
89     """
90     for index in range(len(arr)):
91         if arr[index] == target:
92             return index
93     return -1

```

```

93     return -1
94 # Binary Search implementation for sorted data
95 def binary_search(arr, target):
96     """
97     Perform binary search on a sorted array.
98
99     Parameters:
100     arr (list): The sorted list of elements to search through.
101     target: The element to search for.
102
103     Returns:
104     int: The index of the target element if found, otherwise -1.
105     """
106     left, right = 0, len(arr) - 1
107     while left <= right:
108         mid = left + (right - left) // 2
109         if arr[mid] == target:
110             return mid
111         elif arr[mid] < target:
112             left = mid + 1
113         else:
114             right = mid - 1
115     return -1
116
117 # Sample input for Linear Search
118 unsorted_data = [5, 3, 8, 1, 2]
119 target_linear = 3
120 print("Linear Search:")
121 result_linear = linear_search(unsorted_data, target_linear)
122 print(f"Target {target_linear} found at index: {result_linear}")
123
124 # Sample input for Binary Search
125 sorted_data = [1, 2, 3, 5, 8]
126 target_binary = 3
127 print("Binary Search:")
128 result_binary = binary_search(sorted_data, target_binary)
129 print(f"Target {target_binary} found at index: {result_binary}")
130

```

```

# Explanation:
# Binary Search is applicable only when the data is sorted. It works by repeatedly dividing the search
# space in half, which allows it to find the target element in logarithmic time. The time complexity of Linear Search is O(n) b
|

```

```

Python 3.11.7 Shell
C:\Users\navya\OneDrive\Desktop\AI Assit> python .\12.1.py
Linear Search:
Target 3 found at index: 1
Binary Search:
Target 3 found at index: 2
C:\Users\navya\OneDrive\Desktop\AI Assit>

```

Observation

- Linear Search works on both sorted and unsorted data.
- Binary Search only works when the data is sorted.

- Linear Search checks each element $\rightarrow O(n)$.
- Binary Search divides the list repeatedly $\rightarrow O(\log n)$.
- For large sorted datasets, Binary Search is much faster.

Task 4: Quick Sort vs Merge Sort

Prompt:

Complete recursive Quick Sort and Merge Sort in Python.

Add docstrings and explain recursion.

Compare best, average, and worst-case complexities.

Test on random, sorted, and reverse-sorted data.

```

39 # Quick Sort implementation
40 def quick_sort(arr):
41     """
42     Sort an array using the Quick Sort algorithm.
43
44     Parameters:
45     arr (list): The list of elements to be sorted.
46
47     Returns:
48     list: A new sorted list.
49     """
50     if len(arr) <= 1:
51         return arr
52     pivot = arr[len(arr) // 2]
53     left = [x for x in arr if x < pivot]
54     middle = [x for x in arr if x == pivot]
55     right = [x for x in arr if x > pivot]
56     return quick_sort(left) + middle + quick_sort(right)
57 # Merge Sort implementation
58 def merge_sort(arr):
59     """
60     Sort an array using the Merge Sort algorithm.
61
62     Parameters:
63     arr (list): The list of elements to be sorted.
64
65     Returns:
66     list: A new sorted list.
67     """
68     if len(arr) <= 1:
69         return arr
70     mid = len(arr) // 2
71     left_half = merge_sort(arr[:mid])
72     right_half = merge_sort(arr[mid:])
73     return merge(left_half, right_half)

```

```

174 def merge(left, right):
175     """Merge two sorted lists into a single sorted list."""
176     result = []
177     i = j = 0
178     while i < len(left) and j < len(right):
179         if left[i] < right[j]:
180             result.append(left[i])
181             i += 1
182         else:
183             result.append(right[j])
184             j += 1
185     result.extend(left[i:])
186     result.extend(right[j:])
187     return result
188
189 # Sample input for testing
190 random_data = [3, 1, 4, 1, 5, 9]
191 sorted_data = [1, 1, 3, 4, 5, 9]
192 reverse_sorted_data = [9, 5, 4, 3, 1, 1]
193 print("Quick Sort:")
194 print("Random Data:", quick_sort(random_data))
195 print("Sorted Data:", quick_sort(sorted_data))
196 print("Reverse Sorted Data:", quick_sort(reverse_sorted_data))
197 print("Merge Sort:")
198 print("Random Data:", merge_sort(random_data))
199 print("Sorted Data:", merge_sort(sorted_data))
200 print("Reverse Sorted Data:", merge_sort(reverse_sorted_data))
201
202 # Explanation:
203 # Both Quick Sort and Merge Sort are efficient sorting algorithms with an average and best-case time complexity of  $O(n \log n)$ .
204 # Quick Sort has a worst-case time complexity of  $O(n^2)$  when the pivot is consistently the smallest or largest element.
205 # Merge Sort consistently has a time complexity of  $O(n \log n)$  in all cases.
206 # Quick Sort is generally faster in practice due to better cache locality and lower constant factors, but Merge Sort is stable and guarantees  $O(n \log n)$  in all cases.

```

```

Quick Sort:
Random Data: [1, 1, 3, 4, 5, 9]
Sorted Data: [1, 1, 3, 4, 5, 9]
Reverse Sorted Data: [1, 1, 3, 4, 5, 9]
Merge Sort:
Random Data: [1, 1, 3, 4, 5, 9]
Sorted Data: [1, 1, 3, 4, 5, 9]
Reverse Sorted Data: [1, 1, 3, 4, 5, 9]
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>

```

Observation

- Both algorithms are much faster than Bubble and Insertion for large datasets.
- Quick Sort is usually faster in practical cases.
- Quick Sort worst case $\rightarrow O(n^2)$ (if pivot is bad).
- Merge Sort always guarantees $O(n \log n)$.
- Merge Sort requires extra memory.
- Quick Sort uses less memory (in-place).

Task 5: Duplicate Detection

Prompt:

Write a brute-force duplicate detection algorithm using nested loops.

Analyse its time complexity.

Optimize using set or dictionary.

Compare both approaches and explain performance improvement.

Show sample input/output

```
10 #Show sample input/output
11 def brute_force_duplicate_detection(arr):
12     """Detect duplicates using nested loops."""
13     duplicates = []
14     for i in range(len(arr)):
15         for j in range(i + 1, len(arr)):
16             if arr[i] == arr[j] and arr[i] not in duplicates:
17                 duplicates.append(arr[i])
18     return duplicates
19 def optimized_duplicate_detection(arr):
20     """Detect duplicates using a set."""
21     seen = set()
22     duplicates = set()
23     for num in arr:
24         if num in seen:
25             duplicates.add(num)
26         else:
27             seen.add(num)
28     return list(duplicates)
29 # Sample input
30 sample_data = [1, 2, 3, 2, 4, 5, 3, 6]
31 print("Brute Force Duplicate Detection:")
32 print("Input:", sample_data)
33 print("Duplicates:", brute_force_duplicate_detection(sample_data))
34 print("Optimized Duplicate Detection:")
35 print("Input:", sample_data)
36 print("Duplicates:", optimized_duplicate_detection(sample_data))
```

Explanation:

The brute-force approach has a time complexity of $O(n^2)$ because it uses nested loops to compare each element with every other element.

The optimized approach using a set has a time complexity of $O(n)$ because it only iterates through the array once and uses constant-time lookups.

This results in a significant performance improvement for large datasets.

```
Brute Force Duplicate Detection:  
Input: [1, 2, 3, 2, 4, 5, 3, 6]  
Duplicates: [2, 3]  
Optimized Duplicate Detection:  
Input: [1, 2, 3, 2, 4, 5, 3, 6]  
Duplicates: [2, 3]  
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation

- Brute-force method compares every element with every other element → $O(n^2)$.
- It becomes extremely slow for large datasets.
- Optimized method using set checks elements only once → $O(n)$.
- Set lookups are very fast (constant time).